Kavach: Semantic Pre-Execution Interception for LLM Agent Runtimes

A Measurement, a Deployment, and the Aggregation Frontier

Anonymous Author(s)

ABSTRACT

LLM agents increasingly act in the world through tool calls, but the runtimes that host them provide no point at which a resolved tool invocation can be inspected before it executes — guardrails today watch what an agent says, not what it is about to do. We present Kavach, a runtime monitor that closes this gap by intercepting every tool call at the moment its arguments are resolved, scoring its semantic intent against a curated, taxonomy-grounded corpus of attack patterns before any side effect occurs. We make four contributions. First, a measurement: we show that a widely used agent runtime's gateway path exposes no working pre-execution hook at all, characterize where interception is achievable, and contribute upstream fixes. Second, a deployment: a parliament of four domain-specialized detectors reaches a verdict in real time on production hardware, with every decision grounded in a structured taxonomy and folded into a tamper-evident ledger; we also report an instructive failure — a corpus authored in one linguistic register systematically misjudges attacks phrased in another — and the retrieval fix that recovers from it. Third, the aggregation frontier: we show that combining detector votes by confidence, rather than by veto, is fragile to an attacker who can corrupt even one detector, and we characterize the resulting trade-off between robustness and false alarms. Fourth, a contamination-controlled evaluation: the attack corpus was authored blind to the benchmarks it is tested against, and false-positive rate is measured before — not alongside — attack recall, so that detection numbers cannot be inflated by aggressiveness. Kavach needs no GPU; CPU-only deployment is supported end-to-end, with a GPU only accelerating the embedding step. It is fully reproducible offline.

1 INTRODUCTION

LLM agents have moved from research demos to deployed systems that handle non-trivial work: code generation, infrastructure operations, sales pipelines, and autonomous decision-making. The implementations of these systems have a striking structural property — the agent's harness, the deterministic code that wraps the language model with tool dispatch, context management, and persistence, now constitutes the overwhelming majority of the source lines, while the model itself is a small fraction. The model is no longer the system. The harness is.

This shift has produced a new class of vulnerabilities, exploited at the harness layer rather than at the model. Four incidents from the past six months illustrate the pattern unambiguously.

CVE-2025-59536 and CVE-2026-21852 (Anthropic Claude Code, Oct 2025 and Jan 2026). CVE-2025-59536 allowed code injection in the startup trust dialog, enabling attacker-chosen scripts to run before the user could approve them. CVE-2026-21852 allowed exfiltration of the user's API key via a malicious `ANTHROPIC_BASE_URL` override in `.claude/settings.json` — redirecting traffic to an attacker-controlled server and exfiltrating credentials as a side effect of normal agent operation. CVE-2025-59536 was disclosed on October 3, 2025 and CVE-2026-21852 on January 21, 2026; Check Point Research published a consolidated analysis in February 2026, and both were subsequently patched [12].

CVE-2025-68664 (LangChain Core, disclosed December 2025). A serialization-injection vulnerability in the `dumps()`/`dumpd()` APIs enabled arbitrary class instantiation in trusted namespaces (CVSS 9.3 Critical). The attack surface: `secrets_from_env=True` was the default, meaning environment variables containing credentials could be exfiltrated via a prompt-injection-crafted response that was then deserialized. At disclosure time, LangChain was one of the most widely deployed agent frameworks. A companion vulnerability, CVE-2025-68665, affected the LangChain JavaScript SDK.

CVE-2025-34291 (LangFlow, disclosed October 2025). A CSRF vulnerability on the refresh endpoint led to account takeover and remote code execution via the platform's built-in code-execution feature. This is not a patched historical incident: CrowdSec documented *active exploitation in the wild beginning January 23, 2026* [8]. CISA subsequently added CVE-2025-34291 to its Known Exploited Vulnerabilities catalog on May 21, 2026, with a federal remediation deadline of June 4, 2026.

These four cases share a structure: none of them exploited the language model's reasoning. Each was exploited at the harness layer — configuration parsing, serialization, CSRF handling, environment-variable access — by attacks shaped for that layer. The model was incidental. Multiple security vendors and a U.S. government Cybersecurity Information Sheet documented a sustained wave of MCP-layer disclosures across late 2025 and early 2026 [3, 19], making agent infrastructure one of the fastest-growing attack surfaces in production AI systems.

The gap in existing defenses. Existing guardrails address the model's input and output. LlamaFirewall [6] places PromptGuard 2 at message-input time and AlignmentCheck at chain-of-thought time. AgentSpec [23] intercepts at plan-evaluation time. AGrail [17] and ShieldAgent [5] operate at action-evaluation time. CaMeL [9] constrains capability propagation statically. FIDES [7] tracks information-flow labels.

All of these systems — spatial defenses, in our terminology — evaluate the agent's output or context at a single time-step. None of them fire at the *tool-call boundary*: the moment when a resolved tool invocation, with actual runtime arguments, is about to execute. This is the gap that Kavach fills.

There is a second, subtler gap. Systems that *intend* to fire at the tool-call boundary cannot do so if the host runtime's plugin hook is silently broken. On OpenClaw — a widely adopted, rapidly growing

open-source agent runtime with active weekly releases — two upstream bugs (issues #5513 and #5943) prevent `before_tool_call` from firing at all. Issue #5513: the typed-hook runner snapshots the plugin registry at construction time, so hooks registered during the plugin-load phase are invisible to the runner. Issue #5943: `before_tool_call` is defined and documented as available since v2026.3.28 but is never invoked in `executeToolCalls()`. Until both are fixed, no security plugin on OpenClaw — including Kavach — can be a real guardrail. We contribute fixes for both as the first item of our engineering workstream (PR-1).

Contributions. This paper makes four contributions, the first three forming a coherent systems-and-measurement story and the fourth a focused empirical study.

First, a measurement (§4.1). OpenClaw's gateway execution path exposes no working pre-execution plugin hook: two upstream defects (issues #5513 and #5943) prevent `before_tool_call` from firing, so *no* security plugin can intercept there until they are fixed. Interception is achievable only on the embedded runner. We contribute the upstream fixes (PR-1) and argue that a working pre-execution hook should be a mandatory, tested runtime primitive for agent platforms.

Second, a deployment with taxonomy-grounded detection (§3, §4). The detector is a parliament of four specialized ministers — EXECUTOR (code execution; MITRE ATT&CK T1059, OWASP ASI05), VAULT (credential and identity abuse; T1552, T1539, ASI03), CHANNEL (exfiltration and inter-agent channels; T1567, MITRE ATLAS AML.T0086, ASI07), and NAVIGATOR (trajectory drift; T1083, ASI01) — each querying a disjoint ChromaDB collection of attack-pattern descriptions embedded with BGE [26] and fused with BM25 via reciprocal-rank fusion. A deterministic speaker combines verdicts asymmetrically: any minister BLOCK suffices. Every verdict carries a structured provenance chain — matched pattern → technique ID → tactic → kill-chain stage, referencing MITRE ATLAS and ATT&CK — folded into a SHA-256 hash-chained ledger, so that not only the verdict but its taxonomic *grounding* is tamper-evident. We deploy this on production hardware. An instructive failure surfaces there: a corpus authored in tool-call syntax misjudges attacks phrased in natural language, inflating the false-positive rate, and hybrid retrieval substantially recovers from it.

Third, the aggregation frontier (§5). Combining minister votes by confidence — including a Bayesian variant — is fragile to an adaptive attacker who corrupts even a single minister, whereas a pure-veto aggregator is robust but sacrifices benign utility. We measure this false-positive/robustness trade-off on real minister-vote dumps and characterize a tuned hybrid: a Bayesian decision path with a hard veto floor.

Fourth, contamination-controlled evaluation (§4). The attack-pattern corpus was authored from MITRE ATT&CK technique IDs and OWASP Agentic 2026 categories under a documented eight-rule blind-writing protocol that prohibited reference to any specific CVE or benchmark case. InjecAgent [28] was run cold; false-positive rate on fifty benign sessions was measured *before* attack recall and treated as a <5% gate; cost (latency, embeddings per decision) is reported jointly with detection, following Kapoor et al. [14].

Positioning relative to concurrent work. Concurrent with this work, Microsoft announced MXC (Microsoft Execution Containers, Build 2026), an OS-level policy sandbox for OpenClaw agents, alongside ClawGuard [29], a deterministic rule-based tool-call interceptor achieving 0% attack success rate on AgentDojo under explicit rules. Kavach is complementary to both. MXC reduces blast radius but cannot reason about intent: an agent authorized to read credential files and make HTTP calls can be semantically manipulated into exfiltrating those credentials via indirect injection while remaining within its MXC policy. ClawGuard's 0% AgentDojo result is measured against that benchmark's already-low attack-success baseline (0.6–3.1% on the suites we run), and its automatic rule-derivation component is described as unevaluated future work, so the result reflects hand-authored rules rather than a generalizing detector. ClawGuard's authors acknowledge this gap, noting that content-misleading attacks are difficult to intercept without system-level semantic monitoring [29]; a semantic attack-pattern corpus that requires no per-deployment rule authoring is precisely the generalization gap Kavach's design addresses. Kavach occupies precisely this semantic layer. Three papers published in early 2026 require explicit positioning. DeepContext [4] uses a recurrent network propagating a hidden state over fine-tuned, task-attention-weighted turn embeddings (F1 0.84, sub-20 ms), conceptually adjacent to Kavach's COMPASS oracle; our differentiation is that COMPASS compares agent actions against a fixed declared intent rather than tracking evolving user-turn sequences, and feeds a four-minister parliament. PSG-Agent [25] introduces a four-component guardrail (Plan Monitor, Tool Firewall, Response Guard, Memory Guardian) similar in structure to Kavach's ministers; our differentiation is that PSG-Agent is personalized and collapses oracle and detector roles, while Kavach separates them and applies asymmetric combination. OpenClaw PRISM [15] applies defense-in-depth to the same OpenClaw runtime as Kavach; our differentiation is that Kavach uses a semantic attack-pattern corpus and contributes the upstream hook fixes that make pre-execution interception possible on OpenClaw at all.

Scope and non-claims. We do not claim Kavach prevents every class of agent-side attack. The corpus is curated; novel techniques not yet in published taxonomies will not appear in it. Single-vector cosine similarity has known limits relative to sequence-aware detection [2]; this is a stated future-work direction. The four ministers' detection-failure modes may be statistically correlated; measuring this is a stated future-work item. We do not propose Kavach as a replacement for existing prompt-time or output-time guardrails; we argue (§6) that they enforce at different boundaries along the temporal axis and a comprehensive deployment runs both.

Reproducibility. The corpus (401 attack-pattern descriptions + 100 COMPASS calibration pairs), the parliament service, the OpenClaw plugin and upstream PR-1, the benchmark harness, and the calibration protocol are all open-source and shipped with the paper. The reproducibility checklist in our supplementary material specifies the exact validation sequence — including the benign FPR gate (Step 5) that must pass before the InjecAgent run is meaningful — that produced the numbers reported in §4.

2 BACKGROUND

2.1 The agent infrastructure attack surface

Modern LLM agents are composite systems: a language model provides the reasoning surface, while the harness provides tool dispatch, context management, multi-turn state, and persistence. In production agent deployments the harness now accounts for the overwhelming majority of source lines, with the model contributing only a small fraction. This structural inversion has a direct security consequence: the harness, not the model, is the dominant attack surface.

Agent harnesses expose several classes of latent vulnerabilities. *Configuration-as-code* attacks exploit declarative configuration that the harness loads and executes without sanitization (CVE-2026-21852 on Claude Code's `.claude/settings.json`; CVE-2026-21858 on n8n automation workflows). *Serialization-injection* attacks exploit the harness's serialization/deserialization of tool responses and agent state, enabling arbitrary class instantiation in trusted namespaces (CVE-2025-68664 on LangChain Core, CVSS 9.3). *Hook-bypass* attacks exploit incorrectly wired security hook points in the agent runtime — as with OpenClaw issues #5513 and #5943, where `before_tool_call` is documented as available but never invoked. *CSRF and session* attacks exploit the harness's web surface (CVE-2025-34291 on LangFlow, actively exploited in the wild as of January 2026 per CrowdSec [8]).

These vulnerability classes correspond directly to Kavach's minister structure. EXECUTOR detects code-execution-class attacks (CVE-2025-59536, CVE-2025-34291, CVE-2025-68664). VAULT detects credential-theft-class attacks (CVE-2026-21852, CVE-2025-68664). CHANNEL detects exfiltration-class attacks (CVE-2026-21852 via URL redirection, CVE-2026-21858). NAVIGATOR detects trajectory-manipulation attacks that span multiple tool calls. Section 3 gives the MITRE ATT&CK and OWASP Agentic 2026 taxonomic grounding for each minister.

2.2 Existing guardrails and their enforcement points

The guardrail literature can be organized along two axes: (1) *where* in the agent lifecycle the guardrail enforces, and (2) *what mechanism* it uses to detect violations.

Input-time guardrails. operate on the user's message before the agent processes it. PromptGuard 2 [6] (DeBERTa fine-tune, 19.3 ms on A100, 97.5% recall at 1% FPR on English jailbreaks) and Azure Prompt Shields (Microsoft) are canonical examples. These guardrails cannot detect attacks that arrive via tool outputs, which are the dominant attack vector in prompt-injection benchmarks [28].

Plan-time / chain-of-thought guardrails. operate on the agent's reasoning trace before it commits to an action. AlignmentCheck [6] uses Llama 4 Maverick or Llama 3.3 70B to audit the chain-of-thought, reducing AgentDojo ASR from 0.18 to 0.0289 (an 84% reduction) standalone. DRIFT [16] chains a Secure Planner, Dynamic Validator, and Injection Isolator across the plan representation. These guardrails are powerful but expensive (full LLM inference) and cannot inspect the *resolved arguments* of a tool call, only their declarative intent.

Tool-call-time guardrails. operate at the tool-call boundary. This is Kavach's enforcement point. LlamaFirewall's CodeShield [6] applies Semgrep static analysis to code-returning tool outputs, achieving 96% precision and 79% recall on CyberSecEval3 — but only for code generation, not for general tool calls. ShieldAgent [5] retrieves probabilistic rule circuits for each action trajectory. Task Shield [13] evaluates whether each instruction advances the user's task. None of these systems uses a curated semantic corpus of attack-pattern descriptions at the tool-call level.

Information-flow guardrails. tag data with provenance and capability labels, then enforce policies statically. CaMeL [9] (dual-LLM, custom interpreter, 77% task completion with provable security on AgentDojo), FIDES [7] (multi-agent IFC with confidentiality and integrity labels), and RTBAS [30] (attention-based saliency screener that prevents 100% of policy-violating attacks with under 2% utility loss on AgentDojo) belong to this family. Their strength is provability; their limitation is that they cannot detect attacks whose data provenance is correctly labeled but whose argument content is semantically malicious.

Output-time guardrails. operate after the tool has executed, on the agent's reply. NeMo Guardrails (NVIDIA), Conseca [22], and the post-hoc monitor that Kavach replaces belong here. These are useful for forensics and audit but cannot prevent tool execution.

Drift-detection guardrails. measure whether the agent's trajectory diverges from the user's stated intent across turns. TaskTracker [1] uses activation deltas in white-box models. DeepContext [4] uses a turn-level RNN (F1 0.84, sub-20 ms). Trajectory Guard [2] uses a Siamese Recurrent Autoencoder (AAAI 2026 Trustworthy Agentic AI Workshop). Kavach's COMPASS oracle belongs to this family at the session level: a single cosine comparison between the seeded user intent and each proposed agent action.

Across these categories, the tool-call boundary is the only enforcement point at which the *resolved runtime arguments* of an action are available for inspection before that action executes. Input-time and plan-time guards see only declared intent — the user message or the agent's reasoning trace — not the concrete arguments the call will run with; output-time guards observe the action only after it has executed, too late to prevent it; and information-flow guards enforce data provenance rather than the semantics of the argument content. This is Kavach's enforcement point, and it is structurally unreachable by guards in the other categories: they fire at boundaries where the resolved-argument view does not yet, or no longer, exists.

Table 4 places Kavach in context on five axes: enforcement boundary, mechanism, corpus extensibility, verdict combination, and reported AgentDojo / InjecAgent performance. This table is filled in §4 once benchmark numbers are available.

2.3 Benchmarks and evaluation methodology

The three most widely used agent-security benchmarks are:

InjecAgent [28] — 1,054 prompt-injection test cases from UIUC (2024), structured as 17 user tools × 62 attacker tools, split into direct-harm (30 tools) and data-stealing (32 tools) settings. Each case provides a benign user instruction, a tool the agent would invoke, a malicious payload in the tool's output, and the expected

attacker action. The standard reported metric is Attack Success Rate (ASR) at the harness level. Kavach runs InjecAgent cold (§4).

AgentDojo [10] — 97 user tasks and 629 security test cases across four environments: workspace, Slack, banking, and travel. The standard metrics are ASR (lower is better) and utility score (higher is better for benign tasks). We run AgentDojo cold on the Slack suite (20 user tasks × 5 injection tasks, the "important instructions" attack); results are reported in §5.

A methodological note: running InjecAgent before measuring benign false-positive rate produces precision numbers that conflate detector quality with detector aggressiveness. Our evaluation protocol (§4) gates on FPR <5% on hand-curated benign sessions before running any attack benchmark, following the discipline recommended by Kapoor et al. [14].

2.4 The OWASP and MITRE taxonomic grounding

The two most widely cited threat taxonomies for AI agents as of 2026 are:

MITRE ATT&CK for Enterprise — the standard technique taxonomy, now extended to AI systems via MITRE ATLAS. Kavach's ministers draw patterns from T1059 (Command and Scripting Interpreter), T1546 (Event Triggered Execution), T1552 (Unsecured Credentials), T1539 (Steal Web Session Cookie), T1041 (Exfiltration Over C2 Channel), T1567 (Exfiltration Over Web Service), T1083 (File and Directory Discovery), and others. ShieldNet [27] independently derives 25+ attack types from MITRE ATT&CK for their MCP supply-chain benchmark.

OWASP Top 10 for Agentic Applications 2026 — released December 9, 2025. Ten categories: ASI01 Agent Goal Hijack, ASI02 Tool Misuse & Exploitation, ASI03 Identity & Privilege Abuse, ASI04 Agentic Supply Chain Vulnerabilities, ASI05 Unexpected Code Execution (RCE), ASI06 Memory & Context Poisoning, ASI07 Insecure Inter-Agent Communication, ASI08 Cascading Failures, ASI09 Human-Agent Trust Exploitation, ASI10 Rogue Agents. Kavach's minister mapping: EXECUTOR ↔ ASI05 (Unexpected Code Execution), ASI04 (Agentic Supply Chain Vulnerabilities); VAULT ↔ ASI03 (Identity & Privilege Abuse), ASI09 (Human-Agent Trust Exploitation); CHANNEL ↔ ASI07 (Insecure Inter-Agent Communication), ASI08 (Cascading Failures); NAVIGATOR ↔ ASI01 (Agent Goal Hijack), ASI10 (Rogue Agents).

Using both taxonomies as the authoring source for the attack-pattern corpus (§3.2) ensures that the corpus is grounded in the published threat model of the field rather than in the specific attacks the team happened to have seen.

3 KAVACH ARCHITECTURE

3.1 Overview

Kavach is a runtime guardrail that interposes between an LLM agent orchestrator and its tool-execution layer. Every tool call proposed by the agent is intercepted, evaluated by a Parliament of domain-bound Ministers, and sentenced by a deterministic Speaker before any side-effecting action is permitted to execute. The end-to-end request path is: user intent is seeded; each proposed tool call is embedded once; COMPASS scores intent drift; the semantic router activates the relevant ministers, which retrieve against their corpora in parallel; and the Speaker aggregates their verdicts under the trajectory ceiling before returning BLOCK / ESCALATE / ALLOW.

The system exposes a single HTTP endpoint `POST /hook/parliament` that the OpenClaw plugin (§3.4) calls synchronously in the `pre_tool_call` hook. The endpoint returns a JSON verdict `{decision, confidence, ministers_fired, compass_score, drift_delta}` within the configured latency budget.

3.2 Core Components

Ledger. An append-only SQLite journal (`kavach_parliament.db`) records every evaluation request and its verdict. Each row captures: `session_id`, `turn`, `tool_name`, `action_embedding` (768-d float blob), `verdict`, and `timestamp_utc`. The Ledger is the single source of truth for session state; no in-memory session object exists, making the server horizontally stateless between requests. Session risk score is recomputed from the Ledger at each call via a lightweight SQL aggregate, adding <2 ms overhead.

Semantic Router. On each evaluation request the proposed action text is embedded with `BAAI/bge-base-en-v1.5` (768-d, cosine similarity). The resulting vector is compared against five pre-computed domain centroids stored in `kavach_router_config.json`. Ministers whose centroid cosine similarity exceeds $\theta_{\text{route}} = 0.40$ are activated; COMPASS always runs regardless of routing similarity.

The router replaces the stage-bound dispatcher used in early prototypes (OpenClaw stages Init→Input→Inference→Decision→Execution). Ministers are *domain-bound*, not stage-bound: EXECUTOR fires on any action resembling code execution regardless of which pipeline stage proposed it. This is the key architectural departure from OpenClaw's native trust model and from contemporaneous work such as NemoClaw [21], which enforces a single boundary at the pipeline entry point.

Ministers. Each activated Minister performs a semantic scan: the action embedding is compared via cosine distance against every pattern in the Minister's ChromaDB collection. A hit is declared when $\cos(\mathbf{a}, \mathbf{p}) \geq \theta_{\text{min}}$ where $\theta_{\text{min}} = 0.45$ (Config B; see §4). Each corpus pattern carries L1/L2/L3 descriptors (intent → mechanism → surface) that are propagated into the verdict log for provenance (described in §3.3).

The four domain-bound Ministers and their attack-surface coverage:

- **EXECUTOR** — code execution, process injection, persistence mechanisms, container escape (100 patterns; MITRE ATT&CK T1059, T1055, T1053, T1611 and sub-techniques).
- **VAULT** — credential access, secret file reads, metadata service queries, platform keychain enumeration (100 patterns; T1552, T1528, T1555 and sub-techniques).
- **CHANNEL** — data exfiltration via any outbound channel: HTTP/S, DNS, SMTP, cloud storage, git push, covert channels (100 patterns; T1048, T1567, T1102 and sub-techniques).
- **NAVIGATOR** — session trajectory anomalies: post-task persistence, scope widening, recon sequences, anti-forensic actions (100 patterns; OWASP ASI01, T1083, T1082, T1070).

COMPASS.. The Calibrated Oracle for Mission-Pattern Alignment and Session Semantics evaluates *intent alignment* rather than

attack-pattern similarity. It maintains a *session centroid* — an exponential moving average of action embeddings weighted by turn recency — and measures cosine drift between the centroid and the seeded user-intent vector (extracted from the first non-system turn of the session).

$$\delta_{\text{drift}}(t) = 1 - \cos(\bar{\mathbf{c}}_t, \mathbf{u}) \tag{1}$$

where $\bar{\mathbf{c}}_t$ is the turn-t session centroid and $\mathbf{u}$ is the user-intent embedding. When $\delta_{\text{drift}} > \theta_{\text{drift}}$ the call is escalated regardless of individual minister verdicts.

The COMPASS calibration set (`kavach_corpus_v1.json`, key `COMPASS`) contains 100 labelled pairs — 50 ALIGNED and 50 HIJACKED — covering 25 distinct user-intent scenarios each paired with both a legitimate and an attack-injected proposed action. Youden's J statistic is used to select θ_{drift} via `compass_calibrator.py`.

Speaker. The Speaker is a deterministic finite function — no learned parameters — that maps the set of Minister verdicts and COMPASS output to a single sentence:

$$V = \text{Speaker}(\{v_m\}_{m \in M_{\text{active}}}, \delta_{\text{drift}}, r_{\text{session}}) \tag{2}$$

The decision table (applied in priority order):

(1) Session risk $r_{\text{session}} \geq r_{\max} \Rightarrow$ BLOCK (session-level trajectory ceiling; fires independently of any single Minister's verdict — this is what makes Kavach a session monitor rather than a per-call guard).
(2) Any Minister returns BLOCK $\Rightarrow$ BLOCK.
(3) $\delta_{\text{drift}} > \theta_{\text{drift}} \Rightarrow$ ESCALATE.
(4) Any Minister returns GREY (ESCALATE) $\Rightarrow$ ESCALATE.
(5) All Ministers return ALLOW $\Rightarrow$ ALLOW.

The aggregation is a pure veto: a single BLOCK suffices, with no confidence-weighted averaging. §5 studies this choice empirically against Bayesian and hybrid alternatives under adaptive attack.

Bayesian alternative: priors are estimated, not fixed. The Bayesian aggregator we compare against in §5 does not use hardcoded conditional probabilities. Its prior and per-Minister reliabilities are maintained as Beta–Bernoulli posteriors over ground-truth-labelled outcomes. The block prior is the shrinkage estimate

$$\hat{\pi}_{\text{block}} = \alpha \frac{b}{N} + (1 - \alpha)\, \pi_0, \qquad \alpha = \min\left(\frac{N}{100}, 1\right), \tag{3}$$

where b is the number of BLOCK ground-truth labels among N labelled ledger entries and π_0 a weak default. Each Minister's reliability $r_{m,v}$ for vote type v is the running posterior mean of a Beta$(\cdot)$ updated as

$$r_{m,v} \leftarrow \frac{r_{m,v}\, n_{m,v} + \mathbb{1}[\text{vote correct}]}{n_{m,v} + 1}, \qquad n_{m,v} \leftarrow n_{m,v} + 1. \tag{4}$$

Crucially, these updates are applied *offline* against verified ground truth and never at inference time. This is a deliberate security decision: allowing the runtime to fabricate its own "ground truth" from underdetermined self-decisions would create a feedback channel an adversary could poison to drift the priors. The deployed Speaker is the deterministic veto above; the Bayesian path is an analyzed alternative, not the enforcement path.

3.3 Taxonomy-grounded provenance

Every verdict carries a provenance chain that grounds it in a structured cyber-taxonomy: the matched corpus pattern resolves to a technique identifier (MITRE ATLAS for AI/agent-native techniques, e.g. AML.T0051 indirect prompt injection and AML.T0086 exfiltration via agent tool invocation; MITRE ATT&CK for host-level techniques), which maps to a tactic and an ordered kill-chain stage. Resolution is corpus-tagged where a pattern declares its source technique, with a per-minister default as a fallback so the chain is always populated; the chain records which basis was used, so the audit trail is explicit about precision. The provenance record is serialized into the SHA-256 hash-chained Ledger as a hashed field. Consequently, editing the *grounding* of a historical decision — for instance, relabeling a credential-access block as a benign read to mislead an auditor — breaks the hash chain at that entry, detected by `/ledger/verify`. Prior runtime monitors provide either explainability or tamper-evident logging but not a taxonomy-grounded chain whose integrity is cryptographically verifiable; the ordered stage axis additionally provides the substrate for stage-aware trajectory reasoning, which we leave to future work.

3.4 OpenClaw Plugin

The Kavach plugin (`openclaw-plugin-kavach.ts`) implements the OpenClaw plugin interface:

- **`pre_tool_call`**: synchronously calls `/hook/parliament` before the agent executes a tool. On BLOCK or ESCALATE, the plugin throws `KavachBlockError` or `KavachEscalateError`, preventing tool execution.
- **`on_tool_error`**: increments the circuit-breaker failure counter. After `kavachCircuitBreakerThreshold` consecutive Kavach server errors the plugin enters *open-circuit* mode: all tool calls are blocked and an operator alert is raised.

The plugin is configured via `openclaw.plugin.json` with `parliamentUrl` (default: `http://127.0.0.1:8088`) and `SESSION_ID` in the OpenClaw runtime environment.

We emphasize that Kavach itself is *not* an OpenClaw component: all detection logic — embedding, the four Ministers, the Speaker, and the Ledger — lives in `parliament/server.py`, a standalone ASGI (FastAPI) service exposing HTTP endpoints. The OpenClaw plugin is a ∼150-line adapter that renders a tool call to text and `POST`s it; any agent framework that can issue an HTTP request at a pre-execution hook integrates against the same service without modification. The framework binding is an adapter, not a dependency.

3.5 Design Decisions and Trade-offs

Ministers are domain-bound, not stage-bound. OpenClaw's five-stage pipeline (Init→Execution) provides natural hooks but creates stage-specific blind spots: an attack that appears in the Inference stage looks different from the same attack in the Decision stage, requiring per-stage classifiers. By binding Ministers to attack *domains* we achieve consistent coverage across all stages with a single collection per domain.

Append-only Ledger, no session objects. Storing session state in an append-only log rather than an in-memory object prevents an

attacker from clearing session state via a crafted tool call. The immutability guarantee is enforced at the database level (`kavach_parliament.db` opened read-write only by the server; no `DELETE` or `UPDATE` statements are issued).

Synchronous evaluation on the hot path. Kavach evaluates in the `pre_tool_call` hook synchronously. Asynchronous evaluation would reduce latency but allow a race between the verdict and execution; we accept the latency cost to maintain the security guarantee that *no tool executes before a verdict is issued.*

4 DEPLOYMENT AND DETECTION

4.1 The gateway hook gap

Our first finding is a property of the host runtime, not of Kavach. OpenClaw exposes two execution paths: an *embedded* runner used when the agent is driven programmatically, and a *gateway* path used by the interactive terminal and the hosted gateway. A pre-execution security plugin must run at the `before_tool_call` boundary — after arguments are resolved, before the tool executes. On the gateway path this boundary is not reachable by any plugin: issue #5513 causes the typed-hook runner to snapshot the plugin registry at construction, hiding hooks registered during plugin load, and issue #5943 means `before_tool_call`, although documented as available since v2026.3.28, is never invoked in `executeToolCalls()` on that path. A sentinel-file probe confirms the gap: a tool call that writes a marker executes with no plugin interception on the gateway path, while the same call is intercepted in embedded mode. We contribute upstream fixes for both defects (PR-1) and evaluate Kavach in embedded mode. The general lesson is that a working pre-execution hook is a precondition for *any* runtime guardrail, and agent platforms should treat it as a first-class, tested primitive rather than optional plugin surface.

4.2 Live interception

We evaluate Kavach against an agent running on a Dell Precision 3660 (NVIDIA RTX 4090), with Gemma 4 26B served via Ollama as the agent backbone. Kavach itself comprises its corpus, the four ministers, the Speaker, and the `BAAI/bge-base-en-v1.5` (768-d) embedding model that scores each proposed tool call; on this machine the embedding model is co-located on the GPU for convenience, but it is not GPU-bound (§4.6 runs it CPU-only). A staged attack that induces the agent to read a credential file and transmit its contents is intercepted before execution: the deciding minister returns Block, the call is denied, and a provenance-grounded record (technique → tactic → stage) is committed to the hash-chained Ledger. End-to-end interception latency for the blocking decision is approximately 78 ms. The plugin fails closed on the tool-call path: a verdict that does not return within the timeout denies the call (`toolCallFailMode = deny`).

4.3 Representation mismatch: an instructive negative result

Our corpus is authored in the register of MITRE technique descriptions and tool-call syntax. On InjecAgent [28], whose attacks are expressed as fluent natural-language instructions, this configuration yields a 23.5% hard-block false-positive rate on the direct-harm setting (0.0% on data-stealing) on benign actions evaluated per distinct case — higher than the corpus's tool-call register would suggest, and concentrated almost entirely in one setting. We attribute this elevated false-positive rate to a *representation mismatch*: the corpus is embedded in tool-call/technique register while InjecAgent's benign actions are phrased as natural language, so benign natural-language actions land near attack patterns in embedding space despite sharing no malicious intent. This is an attribution grounded in how the corpus was authored rather than a measured cross-register comparison: a controlled evaluation on benign traffic expressed in the corpus's own tool-call register would be expected to show a markedly lower false-positive rate, and we identify that native-register measurement as a direct test of this hypothesis, left to future work. We report the mismatch because it is the kind of deployment lesson a purely benchmark-simulated evaluation would hide.

4.4 Hybrid retrieval recovers recall at a controlled false-positive rate

The fix follows from the diagnosis. Under dense-only retrieval the lexical channel is absent and benign and attack actions are separated only by embedding proximity; adding the lexical channel both sharpens recall on attacks that share distinctive tokens with the corpus and discounts benign actions that are merely semantically near an attack pattern. We add BM25 over the same corpus, fused with dense retrieval by reciprocal-rank fusion for candidate selection, with the final confidence computed as the dense cosine similarity scaled by a lexical gate (a benign action that is semantically near an attack pattern but shares none of its distinctive tokens is discounted). With this hybrid configuration, on InjecAgent Kavach holds strict (hard-Block) recall at 0.633 (direct-harm) / 0.438 (data-stealing) and loose (block-or-escalate) recall at 0.90 / 0.875. The residual false-positive rate (23.5% hard-block on direct-harm, 0.0% on data-stealing) is a corpus-quality property addressed separately — a small number of over-broad CHANNEL patterns account for it (§7) — rather than a property of the retrieval fusion, which recovers recall. The gate floor is a single tunable parameter; Table [TBD] reports the recall/FPR trade-off across its range.

4.5 Detection results and cost

Table [TBD] reports InjecAgent strict and loose recall and false-positive rate under per-minister thresholds with hybrid retrieval. Per-minister detection rates are uneven, and hard-block detection is carried by EXECUTOR and NAVIGATOR; VAULT and CHANNEL act as escalators on this benchmark (VAULT's per-minister threshold sits above its observed attack-confidence ceiling, so it contributes escalations rather than hard blocks — see §7). §5 reports the minister ablation that motivates the four-way split. On cost, the embed-once optimization reduces embedding calls per decision from up to nine to one, and measured latency is 78 ms p50 / 83 ms p95 — an order of magnitude below lifecycle-instrumenting scanners (PRISM [15] ≈15.8 s p95; AgentArmor [24] ≈20.9 s). We report cost jointly with detection rather than in isolation. All headline detection numbers are cold transfer from third-party benchmarks (InjecAgent [28], AgentDojo [10]) whose attacks the corpus was not authored against; the synthetic generator (`make_synthetic.py`) is

used *only* to validate that the §5 estimator recovers a known injected error-correlation ρ, and never as an evaluation of the detector against its own payloads.

Semantic triage, not a regex fast path. Cost is controlled by a two-stage semantic pipeline rather than a syntactic pre-filter. A single query embedding is computed once per action (the embed-once optimization above) and reused by a lightweight router that gates which Ministers fire: a Minister is activated only when the action's cosine similarity to that Minister's domain descriptors exceeds θ_{route} (0.40), so a typical action runs one or two Ministers rather than all four. This router is Kavach's triage layer. We deliberately do *not* add a zero-LLM regex/AST fast path: the central finding of §4.3 is that attacks expressed in natural-language register evade tool-call/technique-syntax matching, so a syntactic pre-filter would reintroduce exactly the representation mismatch the semantic corpus exists to close, and would silently pass the fluent-language attacks Kavach is designed to catch. A complementary optimization that is *safe* because it never suppresses a detection is a deterministic allowlist for provably benign calls (e.g. workspace-scoped read-only operations) evaluated before embedding; we leave this latency refinement to future work.

Finally, `/ledger/verify` re-walks the hash chain and detects any post-hoc edit to a recorded decision or its provenance, which we demonstrate on a tampered ledger in the appendix.

4.6 Cross-model generalization

The headline numbers above are measured on the primary configuration (Dell Precision 3660, RTX 4090, Gemma 4 26B). To test whether parliament decisions are driven by the shared semantic corpus rather than by a particular agent backbone, we replicate the InjecAgent detection run on a secondary, CPU-only configuration (Intel i5, 16 GB RAM) with `qwen2.5:3b` as the agent backbone, holding the embedding model and corpus identical. On this secondary configuration the parliament catches 26/30 attack actions at the block-or-escalate level (loose recall 0.87) with zero request errors, and on a separate benign gate of 17 natural-language user instructions it hard-blocks *none* (block-only FPR 0.0%); mean per-action latency is ≈ 2.6 s, reflecting CPU-only embedding on this laptop versus the GPU-accelerated embedding available on the primary machine. Strict (hard-Block) recall is markedly lower on the secondary configuration (0.13): under the laptop's freshly calibrated thresholds the operating point shifts toward Escalate on *both* sides—attacks are flagged rather than hard-blocked (strict 0.13 vs. loose 0.87), and 4/17 benign instructions escalate (block-or-escalate FPR 23.5%) while none are blocked. Cross-model agreement is therefore at the level of *flagging* suspicious actions, not of the final block decision, and the hard-Block decisions that do fire remain high-precision (no benign hard-blocks observed). We read this as qualified support for backbone independence—the shared corpus, not the agent model, drives whether an action is flagged—while noting the small benign sample and that a threshold-matched comparison against the primary configuration remains [TBD]. Table [TBD] reports the two configurations side by side; full secondary-run artifacts are in `benchmarks/results_v2/laptop_qwen25_3b/`.

Table 1: Inter-minister error correlation. Average off-diagonal $\rho = 0.091$ is the correlation discount supplied to the Bayesian aggregator.

Minister	CHANNEL	EXECUTOR	NAVIGATOR	VAULT
CHANNEL	1.00	0.05	0.52	0.09
EXECUTOR	0.05	1.00	0.16	-0.01
NAVIGATOR	0.52	0.16	1.00	-0.26
VAULT	0.09	-0.01	-0.26	1.00

5 THE AGGREGATION FRONTIER

The Speaker combines four minister verdicts into one decision. The deployed choice is a pure veto: any single Block blocks. This section asks whether that is the right choice, by measuring it against alternatives under an adaptive attacker. The question matters because the aggregator is the single point at which an attacker who has compromised the agent's reasoning can try to launder a malicious action past the monitor.

5.1 Adaptive attacker model

We model an attacker who, per attack action, fully controls the output of the K most-foolable ministers, forcing each to vote Allow at high confidence. A robust aggregator degrades gracefully as K grows; a brittle one collapses at K=1. The evaluation is a replay: attack-success rate as a function of K and clean false-positive rate on benign sessions are computed from recorded minister votes on real session dumps (no model re-execution), with corrupted votes injected into the replay. This isolates the aggregator's contribution from the ministers' detection quality.

5.2 Veto, Bayesian, and a tuned hybrid

We compare three aggregators. The deployed *pure veto* achieves the lowest attack-success rate at every K — corrupting ministers does not help an attacker unless *all* blocking ministers are corrupted — but its clean false-positive rate is the highest, because any one over-eager minister blocks a benign action ([TBD]% on the benign set). A *Bayesian* aggregator that weights minister votes by learned reliability achieves a low clean false-positive rate ([TBD]%) but is brittle: at K=[*TBD*] corrupted ministers its attack-success rate rises to [TBD]%, because the posterior can be moved by a minority of confident Allow votes. The *hybrid* we propose — a Bayesian decision path with a hard veto floor at a swept confidence threshold — passes the <5% false-positive gate ([TBD]%) while tracking the pure veto's robustness to within [TBD] points at K=[*TBD*]. Table [TBD] reports the full false-positive/robustness frontier as the veto-floor threshold is swept; the contribution is the characterization of this trade-off, not a claim that any single aggregator dominates.

The measured inter-minister error correlation (Table 1, mean off-diagonal $\rho = 0.091$) is the discount the Bayesian aggregator applies when fusing votes; the resulting frontier is reported next.

5.3 Minister ablation

To justify the four-minister decomposition, we ablate ministers and the COMPASS and trajectory signals. Table [TBD] reports detection with one through four ministers, with and without COMPASS,

Table 2: Aggregation frontier: clean false-positive rate and attack-success rate as an adaptive attacker corrupts K ministers. Measured on 2108 replayed actions ($\rho = 0.091$).

Aggregator	Clean FPR	$ASR_{K=0}$	$ASR_{K=1}$	$ASR_{K=2}$	$ASR_{K=3}$	$ASR_{K=4}$
Pure veto (deployed)	11.4%	11.3%	25.8%	35.5%	46.8%	100.0%
Bayesian	5.9%	29.0%	29.0%	35.5%	46.8%	100.0%
Hybrid (veto 0.90)	5.9%	29.0%	29.0%	35.5%	46.8%	100.0%
Max-score	8.5%	11.3%	100.0%	100.0%	100.0%	100.0%
Majority	0.0%	72.6%	96.8%	100.0%	100.0%	100.0%

and with and without the trajectory monitor. The decomposition is justified to the extent that removing a minister lowers recall on the technique classes it covers without a compensating false-positive reduction; we report where this holds and where a minister contributes little on the per-call benchmarks (and why its value appears instead in the staged, multi-step setting).

6 RELATED WORK

7 LIMITATIONS

No head-to-head with ClawGuard on AgentDojo. ClawGuard [29] achieves 0% attack-success rate on AgentDojo under explicit user-authored rules, which Kavach does not currently match. Kavach's contribution is complementary — semantic, embedding-based detection that requires no per-deployment rule authoring and extends across session trajectories — but a fair evaluation should report both systems on the same benchmark. A direct comparison is left to future work pending completion of our AgentDojo run.

Tamper-evident, not tamper-proof. The hash-chained Ledger detects post-hoc edits to any recorded decision or its provenance, but an attacker with write access to the database can recompute the entire chain. Defeating that requires anchoring the chain head to an external append-only medium, which we do not implement.

Provenance precision is partial. Verdict provenance is precise only for corpus patterns that declare a source technique; the remainder resolve to a per-minister default. Coverage on the current corpus is therefore partial, and the audit trail records which basis was used. Raising precision requires tagging the full corpus, an incremental effort.

Curated corpus. The attack-pattern corpus is authored from published taxonomies; novel techniques absent from those taxonomies will not be matched. This is intrinsic to corpus-based semantic detection and motivates the live knowledge-base ingestion we sketch as future work.

Small distinct-N, FPR driver, and an escalate-only minister. Our primary numbers come from InjecAgent's 62 distinct attacker tools (30 direct-harm, 32 data-stealing) replayed across 17 user tasks; the distinct-attack count is modest, so per-minister and per-threshold figures should be read as indicative rather than tightly estimated. Two structural findings from the run: the benign false-positive rate is driven almost entirely by the CHANNEL minister — a small number of over-broad exfiltration patterns account for every benign hard-block — which a corpus-quality pass should target first; and VAULT's per-minister block threshold (0.75) sits above its observed attack-confidence ceiling on this benchmark, so VAULT contributes escalations rather than hard blocks. Inter-minister error correlation is low on average ($\bar{\rho} = 0.09$) but not uniform: the NAVIGATOR–CHANNEL pair reaches $\rho = 0.52$, reflecting overlapping pattern coverage that the aggregation analysis (§5) accounts for. Recalibrating per-minister thresholds and deduplicating the CHANNEL/VAULT corpora are concrete next steps.

Single-vector similarity and the trajectory leg. Per-call detection uses single-vector cosine similarity, which has known limits relative to sequence-aware detection; Trajectory Guard [2] reports that mean-pooling centroid approaches cap at F1 ≤ 0.69 on trajectory detection. Kavach's trajectory monitor uses lightweight context signals rather than a trained sequence model, and the session-risk ceiling requires intent seeding to reach full sensitivity. The session ceiling is currently fixed at an empirically calibrated value (0.50, set just above the observed benign session peak across ten multi-step sessions); modulating it by COMPASS intent drift is a natural extension — the architecture already supports it since `compass_sim` is in scope at the ceiling check — but requires drift-seeded calibration data we leave to future work. A learned, stage-aware trajectory model over the provenance stage axis (§3.3) is the natural next step.

Single backbone and runtime. We evaluate with one agent backbone (Gemma 4 26B) on OpenClaw in embedded mode, without MXC. Cross-model generality and a defense-in-depth evaluation inside an MXC sandbox — the production configuration implied by the Build 2026 Microsoft/OpenClaw partnership — are future work.

Benchmark saturation. InjecAgent and AgentDojo are increasingly saturated on frontier models, which compresses the headroom in which a monitor's benefit is visible. Multi-step staged attacks, which our trajectory monitor targets, are under-represented in current benchmarks and motivate a purpose-built scenario set — itself designed to avoid contaminating the corpus.

8 CONCLUSION

Kavach demonstrates that semantic pre-execution interception is achievable on a production agent runtime, that grounding each verdict in a structured cyber taxonomy makes both the decision and its justification tamper-evident, and that the choice of vote aggregator is a measurable robustness/utility trade-off rather than an implementation detail. The pre-execution hook gap we document, and the upstream fixes we contribute, suggest that pre-execution interception should be a first-class runtime primitive for agent platforms.

Table 3: Minister ablation under the deployed pure-veto Speaker, ministers added strongest-first: does the full parliament beat its strongest single minister? ASR counts a missed attack as one the Speaker ALLOWS (BLOCK and ESCALATE both count as caught, matching the loose-recall convention of §4).

Configuration	ASR	FPR	Accuracy
1 ministers (NAVIGATOR)	21.0%	2.8%	0.881
2 ministers (NAVIGATOR+EXECUTOR)	16.1%	2.8%	0.905
3 ministers (NAVIGATOR+EXECUTOR+VAULT)	11.3%	2.8%	0.929
4 ministers (NAVIGATOR+EXECUTOR+VAULT+CHANNEL)	11.3%	11.4%	0.887

Table 4: Comparison of agent guardrails on five axes. "Boundary" indicates where in the agent lifecycle enforcement fires. "Mechanism" is the primary detection approach. "Ext." indicates whether the detection corpus / rule set is extensible without retraining. "Verdict" indicates how multiple detection signals are combined. AgentDojo ASR and InjecAgent Recall are the best numbers reported by the respective papers; *n/r* = not reported; *ext.* = experimental; † = estimated from paper description. Lower ASR is better; higher Recall is better.

System	Boundary	Mechanism	Ext.?	Verdict	AgentDojo ASR ↓	InjecAgent Recall ↑
PromptGuard 2	input	DeBERTa classifier	no	threshold	7.5%	n/r
AlignmentCheck	CoT	LLM judge	no	threshold	2.89%	n/r
CodeShield	output	Semgrep+regex	yes (rules)	threshold	n/r	n/r
LlamaFirewall (combined)	I/CoT/O	3-layer spatial	partial	joint	1.75%	n/r
AgentSpec	plan	rule DSL	yes	all-pass	>10% blocked	n/r
DRIFT	plan	rule+isolator	partial	sequential	n/r	n/r
Task Shield	tool-call	LLM alignment	no	threshold	22%†	n/r
ShieldAgent	tool-call	rule circuits	yes (docs)	probabilistic	n/r	90.1%
PSG-Agent	tool-call	4-component	partial	cross-turn accum.	n/r	n/r
OpenClaw PRISM	tool-call	middleware checkpts	partial	sequential	n/r	n/r
Kavach (ours)	**tool-call**	**embed. corpus**	**yes (corpus)**	**asymmetric**	**[TBD]**	**[TBD]**
CaMeL	IFC	dual-LLM + taint	no	provable	23% (sec)	n/r
FIDES	IFC	label propagation	no	provable	n/r	n/r
RTBAS	IFC	saliency screener	no	threshold	2% loss only	n/r
AGrail	adaptive	LLM + memory	yes (memory)	cooperative	3.8%–5%	n/r
TaskTracker	session	activation delta	no	binary	n/r	n/r
DeepContext	session	RNN over embeds	no	binary	n/r	n/r
Trajectory Guard	trajectory	Siamese RAE	no	score	n/r	n/r
garak [11]	input (test)	vuln. probes	yes (probes)	per-probe	n/r	n/r
PyRIT [18]	input (test)	red-team orch.	yes (attacks)	*n/a*	n/r	n/r
LLM-Guard [20]	input/output	detector suite	yes (scanners)	threshold	n/r	n/r
DKnownAI Guard	mixed	ensemble	partial	threshold	n/r	96.5%
Lakera Guard	input	classifier	no	threshold	n/r	n/r
AWS Bedrock	input	content filter	no	threshold	n/r	n/r

Asymmetric verdict (Kavach): any single minister returning BLOCK vetoes the call unconditionally; COMPASS drift escalates unless a minister blocks first. No score averaging. See §5 for empirical evaluation of aggregation strategies. **Corpus extensibility** (Kavach): new attack patterns are added as natural-language descriptions, re-embedded into ChromaDB without retraining any model component. **Red-team / scanning tools** are complementary, not competing: garak [11] (NVIDIA) and PyRIT [18] (Microsoft) are *offensive* test harnesses that probe a model/agent for vulnerabilities pre-deployment rather than enforce at runtime, and LLM-Guard [20] is an input/output scanner suite operating at the prompt boundary rather than the tool-call boundary. Kavach is orthogonal: a runtime, tool-call-time enforcement layer that these tools can be used to red-team.

REFERENCES

[1] Sahar Abdelnabi, Aideen Fay, Giovanni Cherubin, Ahmed Salem, Mario Fritz, and Andrew Paverd. 2024. Are you still on track!? Catching LLM Task Drift with Activations. arXiv:2406.00799 [cs.CR] Verify full author list at arxiv.org/abs/2406.00799..

[2] Laksh Advani. 2026. Trajectory Guard – A Lightweight, Sequence-Aware Model for Real-Time Anomaly Detection in Agentic AI. arXiv:2601.00516 [cs.LG] AAAI 2026 Trustworthy Agentic AI Workshop (not main conference). Sole author Laksh Advani. Mean-pooling baseline F1=0.69; Siamese RAE model F1=0.88–0.94..

[3] Agent Wars Security Research. 2026. MCP Security 2026: 30 CVEs in 60 Days — What Went Wrong. https://agent-wars.com/news/2026-03-13-mcp-security-2026-30-cves-in-60-days-what-went-wrong. Documents 30 MCP-layer CVEs disclosed Jan–Feb 2026..

[4] Justin Albrethsen, Yash Datta, Kunal Kumar, and Sharath Rajasekar. 2026. DeepContext: Stateful Real-Time Detection of Multi-Turn Adversarial Intent Drift

in LLMs. arXiv:2602.16935 [cs.AI] [VERIFIED] Authors confirmed from arxiv.org/abs/2602.16935. F1=0.84 (their model); baselines at 0.67..

[5] Zhaorun Chen, Mintong Kang, and Bo Li. 2025. ShieldAgent: Shielding Agents via Verifiable Safety Policy Reasoning. arXiv:2503.22738 [cs.CR]

[6] Sahana Chennabasappa, Cyrus Nikolaidis, Daniel Song, David Molnar, Stephanie Ding, Shengye Wan, Spencer Whitman, Lauren Deason, Nicholas Doucette, Abraham Montilla, Alekhya Gampa, Beto de Paola, Dominik Gabi, James Crnkovich, Jean-Christophe Testud, Kat He, Rashnil Chaturvedi, Wu Zhou, and Joshua Saxe. 2025. LlamaFirewall: An Open Source Guardrail System for Building Secure AI Agents. *arXiv preprint arXiv:2505.03574* (2025). [VERIFIED] Authors confirmed from arxiv.org/abs/2505.03574.

[7] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. 2026. Securing AI Agents with Information-Flow Control. In *Proceedings of the 2026 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. arXiv:2505.23643 Introduces FIDES..

[8] CrowdSec. 2026. CVE-2025-34291 Exploited in the Wild: LangFlow AI Framework Under Fire. https://www.crowdsec.net/vulntracking-report/cve-2025-34291. Active exploitation observed beginning 23 Jan 2026..

[9] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2026. Defeating Prompt Injections by Design. In *Proceedings of the 2026 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. arXiv:2503.18813 [cs.CR]

[10] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*. arXiv:2406.13352 [cs.CR]

[11] Leon Derczynski, Erick Galinkin, Jeffrey Martin, Subho Majumdar, and Nanna Inie. 2024. garak: A Framework for Security Probing Large Language Models. arXiv:2406.11036 [cs.CL] [VERIFIED] NVIDIA. Software: https://github.com/NVIDIA/garak. garak = Generative AI Red-teaming and Assessment Kit..

[12] Aviv Donenfeld and Oded Vanunu. 2026. Caught in the Hook: RCE and API Token Exfiltration Through Claude Code Project Files (CVE-2025-59536). https://research.checkpoint.com/2026/rce-and-api-token-exfiltration-through-claude-code-project-files-cve-2025-59536/. Check Point Research. Disclosed February 2026..

[13] Feiran Jia, Tong Wu, Xin Qin, and Anna Squicciarini. 2025. The Task Shield: Enforcing Task Alignment to Defend Against Indirect Prompt Injection in LLM Agents. arXiv:2412.16682 [cs.CR]

[14] Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. 2024. AI Agents That Matter. arXiv:2407.01502 [cs.LG]

[15] Frank Li. 2026. OpenClaw PRISM: A Zero-Fork, Defense-in-Depth Runtime Security Layer for Tool-Augmented LLM Agents. *arXiv preprint arXiv:2603.11853* (2026). Sole author Frank Li. Latency figures are preliminary per the paper's own framing..

[16] Hao Li, Xiaogeng Liu, Hung-Chun Chiu, Dianqi Li, Ning Zhang, and Chaowei Xiao. 2025. DRIFT: Dynamic Rule-Based Defense with Injection Isolation for Securing LLM Agents. In *Advances in Neural Information Processing Systems*. arXiv:2506.12104

[17] Weidi Luo, Shenghong Dai, Xiaogeng Liu, Suman Banerjee, Huan Sun, Muhao Chen, and Chaowei Xiao. 2025. AGrail: A Lifelong Agent Guardrail with Effective and Adaptive Safety Detection. arXiv:2502.11448 [cs.CR]

[18] Microsoft AI Red Team. 2024. PyRIT: Python Risk Identification Tool for generative AI. https://github.com/Azure/PyRIT. Microsoft red-teaming framework..

[19] National Security Agency. 2026. Model Context Protocol (MCP): Security Design Considerations for AI-Driven Automation. https://media.defense.gov/2026/Jun/02/2003943289/-1/-1/0/CSI_MCP_SECURITY.PDF. NSA Cybersecurity Information Sheet. Report number U/OO/6030316-26 | PP-26-1834..

[20] Protect AI. 2024. LLM Guard: a security toolkit for LLM interactions. https://github.com/protectai/llm-guard. Open-source input/output scanner suite..

[21] Traian Rebedea, Razvan Dinu, Makesh Sreedhar, Christopher Parisien, and Jonathan Cohen. 2023. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. arXiv:2310.10501 [cs.CL] NVIDIA programmable-rails guardrail toolkit..

[22] Lillian Tsai and Eugene Bagdasarian. 2025. Contextual Agent Security: A Policy for Every Purpose. arXiv:2501.17070 [cs.CR]

[23] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. 2025. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. arXiv:2503.18666 [cs.CR]

[24] Peiran Wang, Yang Liu, Yunfei Lu, Yifeng Cai, Hongbo Chen, Qingyou Yang, Jie Zhang, Jue Hong, and Ye Wu. 2025. AgentArmor: Enforcing Program Analysis on Agent Runtime Trace to Defend Against Prompt Injection. *arXiv preprint arXiv:2508.01249* (2025). [VERIFIED] Authors confirmed from arxiv.org/abs/2508.01249.

[25] Yaozu Wu, Jizhou Guo, Dongyuan Li, Henry Peng Zou, Wei-Chieh Huang, Yankai Chen, Zhen Wang, Weizhi Zhang, Yangning Li, Meng Zhang, Renhe Jiang, and Philip S. Yu. 2025. PSG-Agent: Personality-Aware Safety Guardrail for LLM-based Agents. arXiv:2509.23614 [cs.CR]

[26] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. 2024. C-Pack: Packed Resources for General Chinese Embeddings. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*. arXiv:2309.07597 [cs.CL]

[27] Zhuowen Yuan, Zhaorun Chen, Zhen Xiang, Nathaniel D. Bastian, Seyyed Hadi Hashemi, Chaowei Xiao, Wenbo Guo, and Bo Li. 2026. ShieldNet: Network-Level Guardrails against Emerging Supply-Chain Injections in Agentic Systems. arXiv:2604.04426 [cs.AI] [VERIFIED] Authors confirmed from arxiv.org/abs/2604.04426..

[28] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*. 10471–10506. arXiv:2403.02691 [cs.CL]

[29] Wei Zhao, Zhe Li, Peixin Zhang, and Jun Sun. 2026. ClawGuard: A Runtime Security Framework for Tool-Augmented LLM Agents Against Indirect Prompt Injection. *arXiv preprint arXiv:2604.11790* (2026). [VERIFIED] Authors confirmed from arxiv.org/abs/2604.11790.

[30] Peter Yong Zhong, Siyuan Chen, and Others. 2025. RTBAS: Defending LLM Agents Against Prompt Injection and Privacy Leakage. arXiv:2502.08966 [cs.CR] Verify full author list at arxiv.org/abs/2502.08966..